

ui-ux-pro-max

Guide de design complet pour applications web et mobiles. Contient 67 styles, 96 palettes de couleurs, 57 associations de polices, 99 directives UX, et 25 types de graphiques pour 13 stacks technologiques. Base de données interrogeable avec recommandations priorisées.

Prérequis

Vérifier si Python est installé :

```
python3 --version || python --version
```

Si Python n'est pas installé, l'installer selon votre système :

macOS :

```
brew install python3
```

Ubuntu/Debian :

```
sudo apt update && sudo apt install python3
```

Windows :

```
winget install Python.Python.3.12
```

Comment Utiliser Ce Workflow

Lorsque l'utilisateur demande un travail UI/UX (concevoir, construire, créer, implémenter, réviser, corriger, améliorer), suivre ce workflow :

Étape 1 : Analyser les Besoins de l'Utilisateur

Extraire les informations clés de la demande :

- **Type de produit** : SaaS, e-commerce, portfolio, tableau de bord, landing page, etc.
- **Mots-clés de style** : minimaliste, ludique, professionnel, élégant, mode sombre, etc.
- **Industrie** : santé, fintech, gaming, éducation, etc.
- **Stack** : React, Vue, Next.js, ou par défaut `html-tailwind`

Étape 2 : Générer le Design System (OBLIGATOIRE)

Toujours commencer avec `--design-system` pour obtenir des recommandations complètes avec raisonnement :

```
python3 prompts/ui-ux-pro-max/scripts/search.py "<type_produit> <industrie> <mots_clés>" --design-system [-p "Nom du Projet"]
```

Cette commande :

1. Recherche dans 5 domaines en parallèle (produit, style, couleur, landing, typographie)
2. Applique les règles de raisonnement depuis `ui-reasoning.csv` pour sélectionner les meilleures correspondances
3. Retourne un design system complet : pattern, style, couleurs, typographie, effets
4. Inclut les anti-patterns à éviter

Exemple :

```
python3 prompts/ui-ux-pro-max/scripts/search.py "beauté spa bien-être service" --design-system -p "Spa Sérénité"
```

Étape 2b : Persister le Design System (Pattern Maître + Surcharges)

Pour sauvegarder le design system avec récupération hiérarchique entre sessions, ajouter `--persist` :

```
python3 prompts/ui-ux-pro-max/scripts/search.py "<requête>" --design-system --persist -p "Nom du Projet"
```

Cela crée :

- `design-system/MASTER.md` — Source de vérité globale avec toutes les règles de design
- `design-system/pages/` — Dossier pour les surcharges spécifiques aux pages

Avec surcharge spécifique à une page :

```
python3 prompts/ui-ux-pro-max/scripts/search.py "<requête>" --design-system --persist -p "Nom du Projet" --page "dashboard"
```

Cela crée également :

- `design-system/pages/dashboard.md` — Déviations spécifiques à la page du Maître

Comment fonctionne la récupération hiérarchique :

1. Lors de la construction d'une page spécifique (ex: "Checkout"), d'abord vérifier `design-system/pages/checkout.md`

- 2. Si le fichier de page existe, ses règles **surchargent** le fichier Maître
- 3. Sinon, utiliser exclusivement **design-system/MASTER.md**

Étape 3 : Compléter avec des Recherches Détaillées (si nécessaire)

Après avoir obtenu le design system, utiliser des recherches par domaine pour plus de détails :

```
python3 prompts/ui-ux-pro-max/scripts/search.py "<mot_clé>" --domain <domaine> [-n <max_résultats>]
```

Quand utiliser les recherches détaillées :

Besoin	Domaine	Exemple
Plus d'options de style	style	--domain style "glassmorphism sombre"
Recommandations de graphiques	chart	--domain chart "tableau de bord temps réel"
Bonnes pratiques UX	ux	--domain ux "animation accessibilité"
Polices alternatives	typography	--domain typography "élégant luxe"
Structure de landing	landing	--domain landing "hero preuve sociale"

Étape 4 : Directives de Stack (Par défaut : html-tailwind)

Obtenir les bonnes pratiques spécifiques à l'implémentation. Si l'utilisateur ne spécifie pas de stack, **utiliser par défaut html-tailwind**.

```
python3 prompts/ui-ux-pro-max/scripts/search.py "<mot_clé>" --stack html-tailwind
```

Stacks disponibles : **html-tailwind**, **react**, **nextjs**, **vue**, **svelte**, **swiftui**, **react-native**, **flutter**, **shadcn**, **jetpack-compose**

Référence des Recherches

Domaines Disponibles

Domaine	Utilisation	Exemples de Mots-Clés
product	Recommandations par type de produit	SaaS, e-commerce, portfolio, santé, beauté, service
style	Styles UI, couleurs, effets	glassmorphism, minimalisme, mode sombre, brutalisme

Domaine	Utilisation	Exemples de Mots-Clés
typography	Associations de polices, Google Fonts	élégant, ludique, professionnel, moderne
color	Palettes de couleurs par type	saas, ecommerce, santé, beauté, fintech, service
landing	Structure de page, stratégies CTA	hero, hero-centric, témoignage, pricing, preuve sociale
chart	Types de graphiques, recommandations de librairies	tendance, comparaison, timeline, funnel, camembert
ux	Bonnes pratiques, anti-patterns	animation, accessibilité, z-index, chargement
react	Performance React/Next.js	waterfall, bundle, suspense, memo, rerender, cache
web	Directives interface web	aria, focus, clavier, sémantique, virtualisation
prompt	Prompts IA, mots-clés CSS	(nom du style)

Stacks Disponibles

Stack	Focus
html-tailwind	Utilitaires Tailwind, responsive, a11y (DÉFAUT)
react	État, hooks, performance, patterns
nextjs	SSR, routing, images, routes API
vue	Composition API, Pinia, Vue Router
svelte	Runes, stores, SvelteKit
swiftui	Views, State, Navigation, Animation
react-native	Components, Navigation, Listes
flutter	Widgets, State, Layout, Theming
shadcn	Composants shadcn/ui, theming, formulaires, patterns
jetpack-compose	Composables, Modifiers, State Hoisting, Recomposition

Exemple de Workflow

Demande utilisateur : "Créer une landing page pour un service de soins de la peau professionnel"

Étape 1 : Analyser les Besoins

- Type de produit : Service Beauté/Spa

- Mots-clés de style : élégant, professionnel, doux
- Industrie : Beauté/Bien-être
- Stack : html-tailwind (défaut)

Étape 2 : Générer le Design System (OBLIGATOIRE)

```
python3 prompts/ui-ux-pro-max/scripts/search.py "beauté spa bien-être service élégant" --design-system -p "Spa Sérénité"
```

Résultat : Design system complet avec pattern, style, couleurs, typographie, effets et anti-patterns.

Étape 3 : Compléter avec des Recherches Détaillées (si nécessaire)

```
# Obtenir les directives UX pour animation et accessibilité
python3 prompts/ui-ux-pro-max/scripts/search.py "animation accessibilité" -domain ux

# Obtenir des options de typographie alternatives si nécessaire
python3 prompts/ui-ux-pro-max/scripts/search.py "élégant luxe serif" --domain typography
```

Étape 4 : Directives de Stack

```
python3 prompts/ui-ux-pro-max/scripts/search.py "layout responsive formulaire" --stack html-tailwind
```

Ensuite : Synthétiser le design system + recherches détaillées et implémenter le design.

Formats de Sortie

Le flag `--design-system` supporte deux formats de sortie :

```
# Box ASCII (défaut) - meilleur pour affichage terminal
python3 prompts/ui-ux-pro-max/scripts/search.py "fintech crypto" --design-system

# Markdown - meilleur pour documentation
python3 prompts/ui-ux-pro-max/scripts/search.py "fintech crypto" --design-system -f markdown
```

Conseils pour de Meilleurs Résultats

- 1. **Être spécifique avec les mots-clés** - "tableau de bord SaaS santé" > "app"
- 2. **Rechercher plusieurs fois** - Différents mots-clés révèlent différentes perspectives
- 3. **Combiner les domaines** - Style + Typographie + Couleur = Design system complet
- 4. **Toujours vérifier l'UX** - Rechercher "animation", "z-index", "accessibilité" pour les problèmes courants
- 5. **Utiliser le flag stack** - Obtenir les bonnes pratiques spécifiques à l'implémentation
- 6. **Itérer** - Si la première recherche ne correspond pas, essayer différents mots-clés

Règles Courantes pour une UI Professionnelle

Ce sont des problèmes fréquemment négligés qui rendent l'UI non professionnelle :

Icônes & Éléments Visuels

Règle	À Faire	À Éviter
Pas d'emoji comme icônes	Utiliser des icônes SVG (Heroicons, Lucide, Simple Icons)	Utiliser des emojis comme 🍌🚀⚙️ comme icônes UI
États hover stables	Utiliser des transitions de couleur/opacité au hover	Utiliser des transforms scale qui décalent le layout
Logos de marque corrects	Rechercher le SVG officiel depuis Simple Icons	Deviner ou utiliser des chemins de logo incorrects
Taille d'icônes cohérente	Utiliser un viewBox fixe (24x24) avec w-6 h-6	Mélanger différentes tailles d'icônes au hasard

Interaction & Curseur

Règle	À Faire	À Éviter
Curseur pointer	Ajouter <code>cursor-pointer</code> à toutes les cartes cliquables/hoverable	Laisser le curseur par défaut sur les éléments interactifs
Feedback hover	Fournir un feedback visuel (couleur, ombre, bordure)	Aucune indication que l'élément est interactif
Transitions douces	Utiliser <code>transition-colors duration-200</code>	Changements d'état instantanés ou trop lents (>500ms)

Contraste Mode Clair/Sombre

Règle	À Faire	À Éviter
Carte glass mode clair	Utiliser <code>bg-white/80</code> ou opacité plus élevée	Utiliser <code>bg-white/10</code> (trop transparent)
Contraste texte clair	Utiliser <code>#0F172A</code> (slate-900) pour le texte	Utiliser <code>#94A3B8</code> (slate-400) pour le corps de texte

Règle	À Faire	À Éviter
Texte atténué clair	Utiliser #475569 (slate-600) minimum	Utiliser gray-400 ou plus clair
Visibilité des bordures	Utiliser border-gray-200 en mode clair	Utiliser border-white/10 (invisible)

Layout & Espacement

Règle	À Faire	À Éviter
Navbar flottante	Ajouter espacement top-4 left-4 right-4	Coller la navbar à top-0 left-0 right-0
Padding du contenu	Tenir compte de la hauteur de la navbar fixe	Laisser le contenu se cacher derrière les éléments fixes
Max-width cohérent	Utiliser le même max-w-6xl ou max-w-7xl	Mélanger différentes largeurs de conteneur

Checklist Pré-Livraison

Avant de livrer le code UI, vérifier ces éléments :

Qualité Visuelle

- ☐ Aucun emoji utilisé comme icône (utiliser SVG à la place)
- ☐ Toutes les icônes proviennent d'un set cohérent (Heroicons/Lucide)
- ☐ Les logos de marque sont corrects (vérifiés depuis Simple Icons)
- ☐ Les états hover ne causent pas de décalage de layout
- ☐ Utiliser les couleurs du thème directement (bg-primary) pas de wrapper var()

Interaction

- ☐ Tous les éléments cliquables ont cursor-pointer
- ☐ Les états hover fournissent un feedback visuel clair
- ☐ Les transitions sont fluides (150-300ms)
- ☐ Les états focus sont visibles pour la navigation au clavier

Mode Clair/Sombre

- ☐ Le texte en mode clair a un contraste suffisant (4.5:1 minimum)
- ☐ Les éléments glass/transparents sont visibles en mode clair
- ☐ Les bordures sont visibles dans les deux modes
- ☐ Tester les deux modes avant livraison

Layout

- ☐ Les éléments flottants ont un espacement correct depuis les bords

- ☐ Aucun contenu caché derrière les navbars fixes
- ☐ Responsive à 375px, 768px, 1024px, 1440px
- ☐ Pas de défilement horizontal sur mobile

Accessibilité

- ☐ Toutes les images ont un texte alt
- ☐ Les champs de formulaire ont des labels
- ☐ La couleur n'est pas le seul indicateur
- ☐ `prefers-reduced-motion` respecté

Fichiers de Données

Fichier	Contenu	Lignes
<code>data/styles.csv</code>	67 styles UI (glassmorphism, neubrutalism, etc.)	68
<code>data/colors.csv</code>	96 palettes par type de produit	97
<code>data/typography.csv</code>	57 associations de polices Google Fonts	58
<code>data/ux-guidelines.csv</code>	99 règles UX et anti-patterns	100
<code>data/landing.csv</code>	Patterns de landing pages	~30
<code>data/charts.csv</code>	25 types de graphiques	26
<code>data/stacks/*.csv</code>	Bonnes pratiques par stack	Variable

Exemples de Commandes Courantes

```
# Portfolio développeur
python3 prompts/ui-ux-pro-max/scripts/search.py "portfolio personal developer" --design-system -p "Portfolio"

# SaaS Dashboard
python3 prompts/ui-ux-pro-max/scripts/search.py "saas dashboard analytics" --design-system -p "Analytics Pro"

# E-commerce Luxe
python3 prompts/ui-ux-pro-max/scripts/search.py "ecommerce luxury fashion" --design-system -p "Maison Éléance"

# Application Santé
python3 prompts/ui-ux-pro-max/scripts/search.py "healthcare medical app calm" --design-system -p "MediCare"

# Fintech/Crypto
python3 prompts/ui-ux-pro-max/scripts/search.py "fintech crypto trading" --design-system -p "CryptoX"
```



```
# Restaurant/Food  
python3 prompts/ui-ux-pro-max/scripts/search.py "restaurant food delivery"  
--design-system -p "Foodie"
```